Architecture Decision Records (ADR)

Project: Aura Promptly
Version: 1.1
Date: 2026-05-14
Status: Baselined (updated post-review)

Each ADR follows the format: **Context → Options → Decision → Rationale → Trade-offs**

Change Log

Version	Date	Changes
1.0	2026-05-14	Initial baseline
1.1	2026-05-14	ADR-05 updated: pgvector replaces OpenSearch as PoC default; ADR-06 updated: two-layer observability; ADR-09 updated: GCP auth revised to direct OIDC→STS; ADR-11 added: LiteLLM image pinning; ADR-12 added: public repo security

ADR-01: AI Gateway — LiteLLM Proxy on ECS Fargate

Context

The platform needs a single API surface that can route requests to multiple AI providers (Bedrock, OpenAI, Vertex AI), enforce spend budgets, track usage per team, and provide an OpenAI-compatible API so application teams do not need provider-specific SDKs.

Options Considered

Option	Pros	Cons
LiteLLM Proxy (self-hosted ECS Fargate)	Full control; open source; built-in spend tracking; OpenAI-compatible; active community; admin UI	Operational overhead; must manage DB and Redis
LiteLLM Cloud (managed SaaS)	No ops; instant setup	Monthly cost ($49+/month); data leaves AWS
Amazon API Gateway + custom Lambda proxy	AWS-native; low ops	No spend tracking; no admin UI; custom routing logic required
Bedrock Converse API direct	Zero overhead; native AWS	No multi-provider routing; no spend tracking per team; not OpenAI-compatible

Option	Pros	Cons
Azure API Management	Excellent for Azure-native	Adds Azure dependency to hub; not AWS-native

Decision

LiteLLM Proxy on ECS Fargate (self-hosted)

Rationale

- OpenAI-compatible API means application teams use the same `openai` Python SDK regardless of which model is invoked — zero re-tooling for spoke apps
- Built-in spend tracking per team and per model satisfies FR-04/FR-05 without custom code
- Admin UI for governance team satisfies FR-07
- ECS Fargate is serverless compute — no EC2 management, scales to 0 when destroyed
- Self-hosting keeps all request/response data within the AWS account (financial services alignment)
- Free tier and open source — no SaaS subscription cost
- Native Bedrock Agent support via `bedrock/agent/{AGENT_ID}/{ALIAS_ID}` route — no custom adapter needed (resolves OQ-04)

Trade-offs

- Must maintain a PostgreSQL database (Aurora Serverless) and Redis (ElastiCache) for LiteLLM
- Aurora Serverless minimum cost even when paused (~$0.03/hour at 0.5 ACU minimum)
- LiteLLM is a fast-moving open source project — image must be pinned to a `-stable` tag (see ADR-11)

ADR-02: Hub Cloud Provider — AWS

Context

The central AI governance account must be on a single cloud provider. The PoC requires Bedrock (AWS-native), and the organisation has experience with AWS.

Decision

AWS as the central hub cloud provider

Rationale

- Amazon Bedrock is the primary model provider — it lives in AWS
- AWS IAM provides strong cross-account and cross-cloud authentication primitives
- AWS API Gateway + WAF is a mature, cost-effective public API entry point
- ECS Fargate eliminates the need for cluster management

Trade-offs

- Azure and GCP spoke teams must connect over internet (acceptable per C-02)

- AWS Bedrock model availability is region-specific — us-east-1 chosen for broadest coverage

---

ADR-03: Spoke Connectivity — Public HTTPS via API Gateway + WAF

Context

Three spoke accounts (Azure, AWS, GCP) need to call the hub. Options range from fully private (VPN/ExpressRoute) to fully public (internet HTTPS).

Options Considered

Option	Security	Cost	Complexity
API Gateway + WAF (HTTPS/internet)	High (WAF + auth + TLS)	~$5/month WAF	Low
AWS Transit Gateway + VPN	Very high	$75+/month	High
AWS PrivateLink	Very high (Azure: needs VPN first)	$50+/month	Very high
Internet API Gateway (no WAF)	Medium	~$0	Very low

Decision

Public HTTPS via API Gateway with AWS WAF

Rationale

- Lowest cost option that meets security requirements
- WAF provides OWASP protection and rate limiting without private networking
- Authentication (API key / OIDC / IAM) adds a second layer of security
- TLS 1.2+ ensures encryption in transit
- Azure and GCP native services (ACA, Cloud Run) can call HTTPS endpoints with zero additional networking setup
- Meets C-02 (no private links required)

Trade-offs

- Traffic traverses public internet (acceptable: encrypted + authenticated)
- WAF adds $5/month — toggle with `enable_waf` Terraform variable to save cost during development
- API Gateway regional endpoint not behind a CDN — latency varies by region (acceptable for PoC)

ADR-04: Bedrock Model Selection — Claude Haiku 3

Context

The PoC needs a capable foundation model for inference, RAG and agentic tasks at minimum cost.

Options Considered

Model	Input $/1K tokens	Output $/1K tokens	Capability
Claude Haiku 3	$0.00025	$0.00125	Good for chat, RAG, summarisation
Claude Sonnet 3.5	$0.003	$0.015	Better reasoning; 12x more expensive
Claude Opus 3	$0.015	$0.075	Best; 60x more expensive
Amazon Titan Text G1 Lite	$0.00015	$0.0002	Limited capability for complex tasks
Llama 3.1 8B Instruct	$0.00022	$0.00022	Open weights; good for simple tasks

Decision

Claude Haiku 3 as primary model
Placeholder Terraform blocks for Claude Sonnet, Llama 3, Mistral included as commented code.

Rationale

- Best capability-to-cost ratio for PoC tasks (chat, RAG, summarisation, agent planning)
- Supported by Bedrock Knowledge Base and Bedrock Agent
- Sufficient quality for demo/educational purposes
- 10,000 tokens ≈ $0.015 — entire PoC session costs cents

Trade-offs

- Lower quality than Sonnet/Opus for complex reasoning tasks
- Not suitable for production use cases requiring high accuracy

---

ADR-05: Vector Store — Bedrock Knowledge Base + Aurora Serverless (pgvector)

Context

RAG requires a vector database for storing and querying document embeddings. The original design used OpenSearch Serverless, but this was reviewed post-baseline due to cost concerns.

Options Considered

Option	Idle cost	Active cost	Managed?	AWS-native?
Bedrock KB + OpenSearch Serverless	~$172/month (2 OCU min, always billing)	~$172/month	Fully managed	Yes

Option	Idle cost	Active cost	Managed?	AWS-native?
Bedrock KB + Aurora Serverless (pgvector)	**~$0 (scale-to-zero)**	**~$10–15/month**	Partially managed	Yes
Pinecone (external)	Free tier → $70+/month	$70+/month	Fully managed	No
Self-managed pgvector on RDS	~$15/month	~$15/month	Manual	Yes
Weaviate Serverless	Free tier → $25+/month	$25+/month	Managed	No

Decision

Bedrock Knowledge Base with Aurora Serverless (pgvector) for the PoC
OpenSearch Serverless documented as the production-scale alternative.

Rationale

- Aurora Serverless pgvector is **GA and fully supported** as a first-class Bedrock Knowledge Base vector store (confirmed May 2026)
- Aurora Serverless scales to 0 ACUs when idle — effectively $0 cost between sessions
- Aurora is already in the stack for LiteLLM; the KB can use a separate schema on the same cluster, saving one resource
- ~90% cost reduction vs OpenSearch Serverless for PoC-scale workloads (few thousand chunks, low QPS)
- AWS provides a CloudFormation quick-create option for Aurora pgvector + Bedrock KB setup
- Titan Text Embeddings v2 (1024 dimensions) is fully supported with pgvector HNSW indexes

Trade-offs

- pgvector HNSW index performance degrades at very high QPS (>1000 req/s) — not relevant for PoC
- OpenSearch Serverless remains the better choice for production-scale RAG (sub-100ms P99 at high QPS)
- Aurora pgvector requires pre-creating the vector table and HNSW index before Bedrock KB can use it (handled by Terraform)

Production guidance (for article/video): At production scale, migrate to OpenSearch Serverless or a dedicated vector DB. The Bedrock KB API is identical — only the backing store changes.

Cost mitigation: Terraform `enable_knowledge_base = true/false` toggle destroys the KB schema when not in use. Aurora cluster itself remains (shared with LiteLLM DB).

ADR-06: Observability Stack — Two-Layer: CloudWatch-Native + Optional OTel → Grafana Cloud

Context

The platform needs centralised observability for requests, costs, latency, and guardrail events. The original design used Grafana Cloud as the sole dashboard layer, but this creates a data egress concern for financial services readers.

Options Considered

Option	Cost	Data egress	Capability
CloudWatch only	~$3/month	None	Good; no traces; AWS-only
CloudWatch (primary) + OTel → Grafana Cloud (optional)	**$0 extra (free tier)**	**Optional, toggleable**	**Full; vendor-neutral**
OTel → Grafana Cloud only	$0 (free tier)	Always	Full; no AWS-native fallback
Datadog	$15+/agent/month	Always	Excellent; expensive
AWS X-Ray + CloudWatch Container Insights	~$5/month	None	AWS-native traces

Decision

Two-layer observability:

1. **Layer 1 (always-on, no egress):** CloudWatch Logs + CloudWatch Metrics (EMF) + CloudWatch Dashboards
2. **Layer 2 (optional demo layer):** OTel Collector ECS sidecar → Grafana Cloud (Loki + Tempo + Prometheus), controlled by `enable_grafana_export = true/false`

Rationale

- CloudWatch is the primary store — all logs and metrics are available regardless of Grafana setting
- Financial services readers can disable `enable_grafana_export` and have a fully compliant, zero-egress observability stack
- Grafana Cloud remains the demo/YouTube layer — visually compelling dashboards, free tier sufficient
- OTel pipeline is vendor-neutral: the same collector config can target Datadog, Splunk, or Azure Monitor by changing the exporter endpoint — this is the key architectural lesson for the article
- LiteLLM supports both CloudWatch (via stdout JSON → CW Logs) and OTel callbacks natively

Trade-offs

- CloudWatch Dashboards are less visually polished than Grafana for YouTube content — mitigated by keeping Grafana as the demo default
- Two layers add slight configuration complexity — mitigated by the Terraform toggle
- Grafana Cloud free tier log retention is 14 days (sufficient for PoC sessions)

Article/video note: Explicitly call out that `enable_grafana_export = false` is the recommended setting for regulated environments. Show both dashboards side-by-side to demonstrate the OTel portability

story.

ADR-07: Terraform State — Terraform Cloud Free Tier

Context

Terraform state must be stored remotely for a team workflow, but the PoC is developed by a single engineer on a local machine.

Options Considered

Option	Cost	Pros	Cons
Terraform Cloud (free)	$0	Remote state, runs, policy checks	1 user limit; 500 resources max
S3 + DynamoDB	~$1/month	AWS-native	Manual setup; no UI
Local state	$0	Zero setup	Not shareable; risky

Decision

Terraform Cloud free tier

Rationale

- Free for single-user use case (matches PoC constraint)
- Provides state locking, state history, and a UI to review plans
- GitHub Actions integration is built-in — no S3/DynamoDB Terraform required
- 500 managed resources is sufficient (PoC target is ~150–200 resources)
- Terraform Cloud free tier supports unlimited workspaces in a single organisation (OQ-02 resolved)

Trade-offs

- If the PoC grows beyond 1 user or 500 resources, upgrade to Terraform Cloud Plus ($20/user/month)
- Requires a Terraform Cloud account (free sign-up)
- Terraform Cloud API token stored in GitHub Secrets (acceptable — not a cloud credential)

ADR-08: Sample Application Framework — Python FastAPI + Streamlit

Context

Three sample applications (Azure, AWS, GCP) need to demonstrate AI consumption patterns and be simple enough for demo purposes.

Decision

Python 3.11 + FastAPI (backend) + Streamlit (UI)

Rationale

- Python is the lingua franca of AI/ML teams — maximises audience for the article/video
- `openai` Python SDK works directly with LiteLLM's OpenAI-compatible endpoint — minimal spoke-side code
- Streamlit produces visually appealing UIs in ~20 lines of code — ideal for demo
- FastAPI is async-native — handles streaming responses efficiently
- Single language across all three spokes keeps the cognitive load low

Trade-offs

- Streamlit not production-grade for real UIs (acceptable for PoC/demo)
- Python cold start on Lambda (~300ms) acceptable for AWS spoke demo use case

---

ADR-09: Authentication Architecture — Revised Layered Approach

Context

Three different cloud providers have different identity systems. The hub must support all three without requiring a VPN. The original design used Cognito OIDC federation for GCP, but this was revised to use direct AWS STS federation, which is simpler and better documented.

Decision

Three complementary auth methods, all terminated at API Gateway:

1. **API Key** (Azure spoke) — simple, effective, stored in Azure Key Vault
2. **IAM Signature V4 via cross-account STS** (AWS spoke) — leverages native AWS identity
3. **IAM Signature V4 via GCP Workload Identity OIDC → AWS STS** (GCP spoke) — direct federation, no Cognito

GCP Auth Flow (revised)

```
Cloud Run service account
  └── GCP metadata server issues OIDC identity token
        └── AssumeRoleWithWebIdentity (AWS STS)
              └── Temporary AWS credentials (15-min TTL)
                    └── AWS Signature V4 call to API Gateway
                          └── API Gateway IAM authoriser validates
```

IAM trust policy for the GCP-federated role:

```
{
  "Effect": "Allow",
  "Principal": { "Federated": "accounts.google.com" },
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "accounts.google.com:sub": "<GCP_SERVICE_ACCOUNT_UNIQUE_ID>"
```

```
      }
    }
  }
```

Rationale

- Direct OIDC → STS federation is the AWS-documented pattern for GCP workloads (AWS Security Blog, Dec 2023)
- Eliminates the Cognito token exchange step — lower latency, fewer moving parts
- Both AWS and GCP spokes now use IAM Sig V4 — single API Gateway IAM authoriser handles both
- GCP Cloud Run service accounts issue OIDC tokens from the metadata server with no additional configuration
- No long-lived credentials anywhere in the GCP spoke

Trade-offs

- Requires creating an AWS IAM OIDC provider for `accounts.google.com` (one-time setup, Terraform-managed)
- The `sub` claim (service account unique ID) must be known before Terraform apply — retrieve with `gcloud iam service-accounts describe`
- Cognito is still used for API key issuance (Azure spoke) — it is not removed from the stack

Comparison with original design

Aspect	Original (Cognito OIDC)	Revised (Direct STS)
Token exchange steps	2 (GCP → Cognito → JWT)	1 (GCP → STS)
Latency overhead	~100–150ms	~50ms
AWS services required	Cognito + API GW	STS + API GW IAM authoriser
Complexity	Medium	Low
AWS documentation	Limited	Official AWS Security Blog

ADR-10: CI/CD Platform — GitHub Actions with OIDC (Phased)

Context

All pipelines (Terraform plan/apply/destroy, smoke tests, Infracost) need a CI/CD platform. GitHub Actions was specified. The original design deferred all CI/CD to Phase 11, but this left the project without any pipeline for the first 10 phases.

Decision

GitHub Actions with OIDC identity federation, split into two phases:

Phase 1b — Basic pipeline (immediately after Phase 1):

- `terraform fmt -check`, `terraform validate`, `terraform plan` on PRs

- `terraform apply` on merge to main (hub workspace)
- AWS OIDC only

Phase 11 — Full pipeline:

- Infracost diff on PRs
- tfsec/checkov security scan
- gitleaks secret scanning
- All-workspace plan/apply
- Azure and GCP OIDC
- Destroy workflow (workflow_dispatch)
- KB sync workflow

Rationale

- Having a basic pipeline from Phase 1 prevents infrastructure drift and catches Terraform errors early
- Infracost and security scanning are valuable but not blocking for early phases
- OIDC to Azure and GCP requires those accounts to exist (Phase 6/7 prerequisites)
- GitHub Actions free tier provides 2,000 minutes/month for public repos (unlimited for public repos — no budget concern)

Trade-offs

- Phase 1b pipeline only covers the hub workspace — spoke workspaces added in Phase 11
- Two-phase CI/CD adds a small amount of pipeline maintenance overhead

---

ADR-11: LiteLLM Docker Image — Pinned Stable Tag with Cosign Verification

Context

LiteLLM is a fast-moving open source project. The `main-latest` tag has caused breaking changes in the past. The project now has a formal release process with signed images.

Decision

Pin to a specific `-stable` semver tag; verify with cosign in CI/CD

```
# infra/hub/modules/litellm/main.tf
image = "ghcr.io/berriai/litellm:v1.83.0-stable"
```

Rationale

- `-stable` tags undergo 12-hour load tests before publishing — significantly more reliable than `main-latest`
- LiteLLM signs all Docker images with cosign from v1.83.0 onwards — integrity can be verified
- Immutable release tags (introduced in CI/CD v2) prevent tampering after publication
- Pinning to a semver enables deliberate, reviewed upgrades via PR

Upgrade process

```
# To upgrade LiteLLM version:
# 1. Check latest stable release at
https://github.com/BerriAI/litellm/releases
# 2. Update image tag in Terraform
# 3. Review release notes for breaking changes
# 4. Open PR — CI pipeline will plan and validate
# 5. Merge after review
```

Cosign verification in CI/CD

```
# .github/workflows/terraform-deploy.yml
- name: Verify LiteLLM image signature
  run: |
    cosign verify \
      --key
https://raw.githubusercontent.com/BerriAI/litellm/0112e53046018d726492c814
b3644b7d376029d0/cosign.pub \
      ghcr.io/berriai/litellm:v1.83.0-stable
```

Trade-offs

- Must manually bump the version tag to get new features or bug fixes
- `-stable` releases lag behind `main` by ~1 week — acceptable for a PoC

ADR-12: Public Repository Security Controls

Context

The GitHub repository is public (educational content for LinkedIn/YouTube). This requires explicit controls to prevent accidental secret exposure in Terraform outputs, config files, and commit history.

Decision

Multi-layer secret prevention strategy:

1. **Terraform outputs:** All sensitive outputs marked `sensitive = true`; secret values never output — only ARNs
2. **Repository settings:** GitHub secret scanning + push protection enabled
3. **CI/CD scanning:** gitleaks or trufflehog on every PR
4. **LiteLLM config:** All credentials referenced via `os.environ/SECRET_NAME` — no literal values in YAML
5. **`.gitignore`:** Covers `*.tfvars`, `terraform.tfstate*`, `.terraform/`, `override.tf`
6. **`CODEOWNERS`:** Requires review for all `infra/` changes

7. `SECURITY.md`: Documents that this is a PoC with no real credentials and provides responsible disclosure contact

Rationale

- Public repos are indexed by secret scanning bots within minutes of a push — prevention is the only viable strategy
- GitHub's native secret scanning is free and catches 200+ credential patterns automatically
- Push protection blocks the commit before it lands — no need to rotate credentials after the fact
- `sensitive = true` in Terraform prevents secrets appearing in Terraform Cloud UI, plan output, and apply output
- `os.environ/` references in LiteLLM config mean the YAML file is safe to commit

Trade-offs

- `sensitive = true` outputs cannot be read with `terraform output` without `-json` flag and explicit acknowledgement — minor operational friction
- gitleaks/trufflehog adds ~30 seconds to PR pipeline — acceptable